

UNIVERSIDAD MAYOR REAL Y PONTIFICIA
DE SAN FRANCISCO XAVIER
FACULTAD DE CIENCIAS Y TECNOLOGIA



**Clasificación Automática de
Billetes**

Mediante Visión por Computadora

Universitario (a): Quispe Torrez Ever

Carrera: Ing. De Sistemas

Docente: Ing. Pacheco Lora Carlos Walter

Materia: Inteligencia Artificial I

Paralelo: SIS420

Fecha: 28/11/2025

Contenido

1. Introducción	1
2. Creación de dataset de Billetes Bolivianos	1
2.1. Data Augmentation (aumento de datos)	2
2.2. Conversión a 64x64 pixeles.....	3
3. Creación del Dataset	4
3.1. Verificación del Dataset.....	4
4. Modelo de Clasificación de Billetes (notebook)	5
4.1. Tecnologías y Bibliotecas Clave	6
4.2. Arquitectura y Configuración del Modelo.....	6
4.3. Detalles del Proceso de entrenamiento	7
4.4. Código utilizado para el Modelo de Clasificación y enlaces	8
5. Modelo YOLOv8n	12
5.1. Label Studio.....	12
5.2. Notebook Colab personalizado	13
5.3. Explicación de como se hizo el modelo YOLO	14
6. DroidCam.....	14
6.1. Código utilizado.....	15
7. Identificador.....	17
8. Enlaces del proyecto.....	24

1. Introducción

Proyecto: Clasificación automática de billetes mediante visión por computadora.

Este proyecto tiene como objetivo desarrollar un sistema capaz de identificar y clasificar billetes bolivianos a partir de imágenes capturadas por una cámara. La idea es colocar los billetes sobre una superficie plana, tomar una fotografía y, mediante técnicas de procesamiento de imágenes y aprendizaje automático, reconocer la denominación correspondiente de cada billete.

Dado que no es posible obtener múltiples imágenes distintas de cada billete real, se aplica la técnica de data augmentation (aumento de datos) para generar variaciones artificiales del dataset original. Esto permite entrenar un modelo más robusto y generalizable, capaz de reconocer los billetes en diferentes condiciones de iluminación, ángulos y escalas.

2. Creación de dataset de Billetes Bolivianos

Para la creación del dataset se tomaron billetes con denominación de 10, 20, 50, 100 y 200 Bs. Estos de la Seria A, Serie B y antigua, ejemplo





2.1. Data Augmentation (aumentación de datos)

Se aplicó la técnica de **Data Augmentation (aumentación de datos)** para generar variaciones artificiales de imágenes para ampliar el conjunto billetes para crear el dataset.

Cálculo del total de imágenes generadas

Cantidad por billete (anverso + reverso):

- 200 imágenes aumentadas por el anverso
- 200 imágenes aumentadas por el reverso
- Total, por billete = 400 imágenes

Tipos de billetes:

5 tipos distintos

- $400 \times 5 = 2\,000$ imágenes por serie

Series (A, B y C):

3 series, cada una con sus 5 tipos de billetes

- $2\,000 \times 3 = 6\,000$ imágenes en total

Se realizó un proceso de aumentación de datos usando una máscara: cada billete, tanto anverso como reverso, se colocó sobre un fondo negro y se marcó la zona del billete con una máscara binaria. Después se aplicaron transformaciones como rotaciones, flips, cambios de perspectiva y escala, además de ajustes de color (brillo, contraste, saturación, tono) y algo de ruido controlado (gaussiano y desenfoque por movimiento). Se corrigieron los artefactos para que el fondo quedara negro puro. Con este pipeline se generaron 200 variaciones por cada cara de billete, y al

multiplicar por 5 tipos de billetes y 3 series, se obtuvo un dataset aumentado de 6000 imágenes en RGB de tamaño 2240×2240.

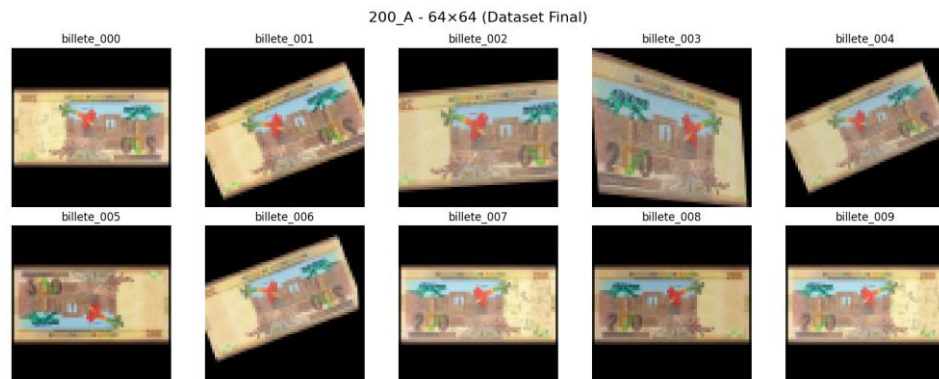
- **imágenes generadas**



2.2. Conversión a 64x64 píxeles

Una vez generado las imágenes grandes, todas se redimensionaron y normalizaron a 64×64 en formato RGB. Con esto se logró que todas tengan el mismo tamaño, se mantuvo el fondo negro limpio y quedaron listas para usarse en modelos de visión computacional. Al final se obtuvo un conjunto de datos de 6000 imágenes en 64×64, para crear el dataset de billetes.

- **Imágenes redimensionadas a 64x64 píxeles**



- **Código utilizado**

Drive

https://drive.google.com/file/d/1gINz7Pitch_RCNrCIU8_fMJmSknEtU0/view?usp=sharing

GitHub

https://github.com/everqt/sis420-ia/blob/main/proyecto_billetes/generar_dataset/generate_billetes.ipynb

3. Creación del Dataset

Se armó un dataset en CSV a partir de las imágenes de billetes ya aumentadas y normalizadas en 64×64 RGB. Cada imagen anverso y reverso. Se asignó un identificador de clase (label) para cada billete:

Clase 0 → 10 Bs

Clase 1 → 20 Bs

Clase 2 → 50 Bs

Clase 3 → 100 Bs

Clase 4 → 200 Bs

Con un total 5 clases en los cuales se podrá clasificar los billetes.

3.1. Verificación del Dataset

Se diseñó un script de verificación para el dataset de billetes en CSV. Primero revisa la estructura: que haya exactamente 12 289 columnas (1 de label + 12 288 de píxeles). Luego comprueba las etiquetas (labels) para confirmar que están en el rango correcto y que cada clase de billete tiene la cantidad esperada de imágenes. Después valida los valores de píxeles, asegurando que estén dentro del rango RGB (0–255) y que no existan nulos. También reconstruye algunas imágenes desde el CSV para verlas en pantalla y confirmar que se pueden recuperar correctamente.

```
... INICIANDO VERIFICACIÓN DEL DATASET CSV
=====
DIMENSIONES: (6000, 12289)
Número de imágenes: 6000
Características por imagen: 12288 píxeles
=====
1. VERIFICACIÓN DE ESTRUCTURA
=====
[✓] Columnas esperadas: 12289
[✓] Columnas encontradas: 12289
[✓] Estructura correcta: True

[■] Primeras 5 columnas: ['label', 'pixel_1', 'pixel_2', 'pixel_3', 'pixel_4']
[■] Últimas 5 columnas: ['pixel_12284', 'pixel_12285', 'pixel_12286', 'pixel_12287', 'pixel_12288']
=====
2. VERIFICACIÓN DE ETIQUETAS
=====
[✓] Etiquetas únicas: [0 1 2 3 4]
[✓] Rango de etiquetas: 0 a 4
```



- **Código utilizado**

Drive

https://drive.google.com/file/d/17jrUIAeAXhtND0yKkuvPy9yA_tuV4OVe/view?usp=sharing

GitHub

https://github.com/everqt/sis420-ia/blob/main/proyecto_billetes/generar_dataset/creacion_dataset.ipynb

4. Modelo de Clasificación de Billetes (notebook)

El objetivo principal del archivo es crear y entrenar un clasificador de imágenes que pueda identificar correctamente a cuál de los 5 tipos distintos de billetes pertenece una imagen. Para lograrlo, se emplea una Red Neuronal Artificial Profunda, una técnica clave dentro del Deep Learning.

El proceso transforma los datos de imagen (píxeles) en un vector numérico de 12,288 características y entrena un modelo para aprender a mapear este vector a una de las cinco etiquetas de clase (los tipos de billetes).

4.1. Tecnologías y Bibliotecas Clave

El notebook se basa en varias librerías de Python, siendo PyTorch la columna vertebral del proyecto

Categoría	Biblioteca / Herramienta	Función Específica en el Archivo
Framework de Deep Learning	PyTorch (torch, torch.nn)	Es el framework principal. Se usa para definir la arquitectura de la red neuronal, manejar tensores (vectores y matrices de datos), configurar la función de pérdida y el optimizador, y ejecutar el proceso de entrenamiento.
Análisis y Manipulación de Datos	Pandas (pd)	Se utiliza para cargar y leer el archivo de datos (billetes_pixels_dataset.csv), que está en formato tabular (CSV).
Cálculo Numérico	NumPy (np)	Se usa para operaciones de cálculo eficientes, especialmente para manejar los píxeles como grandes arreglos numéricos (arrays).
Procesamiento de Datos	Scikit-learn (sklearn)	Se utiliza la función train_test_split para realizar la división estratificada de los datos en conjuntos de entrenamiento (80%) y prueba (20%). También se usa para calcular el Accuracy (precisión) final.

4.2. Arquitectura y Configuración del Modelo

El modelo entrenado es una Red Neuronal de Perceptrón Multicapa (MLP) o también conocida como Red Totalmente Conectada (Fully Connected Network).

Modelo: BilletesMLP (Red Totalmente Conectada) se caracteriza por ser profundo (deep), ya que consta de 8 capas ocultas:

Capa	Tipo	Tamaño de Entrada	Tamaño de Salida	Propósito
Capa de Entrada	nn.Linear	12,288	8,192	El dato de entrada es el vector de píxeles (64x64x3 = 12,288).

Capas Ocultas (8)	nn.Linear	Varias	4096 a 64	Encargadas de extraer características complejas y reducir la dimensionalidad.
Capa de Salida	nn.Linear	64	5	Produce 5 valores de salida (uno para cada clase de billete).

12,288 dimensiones: Proviene de transformar la imagen de 64 píxeles de alto por 64 píxeles de ancho con 3 canales de color (RGB) en una sola lista (vector).

Activación ReLU: La función Rectified Linear Unit (ReLU) se aplica a todas las capas ocultas para introducir no linealidad, lo cual permite que la red aprenda relaciones complejas en los datos de imagen.

4.3. Detalles del Proceso de entrenamiento

El proceso para ajustar los pesos internos de la red neuronal y optimizar su rendimiento es el siguiente:

- 1. Función de Pérdida (Loss Function):** Se utiliza `nn.CrossEntropyLoss`. Esta es la función estándar en PyTorch para problemas de clasificación multiclase. Mide la diferencia entre la distribución de probabilidad predicha por la red y la etiqueta real del billete.
- 2. Optimizador (Optimizer):** Se emplea el optimizador `torch.optim.Adam`. Adam es un algoritmo de optimización muy popular y eficiente que ajusta dinámicamente la tasa de aprendizaje para cada parámetro, acelerando la convergencia del entrenamiento.
- 3. Hiperparámetros de Entrenamiento:**
Épocas: El modelo se entrena durante 2,000 épocas (cada época es un ciclo completo de entrenamiento sobre todo el conjunto de datos).

Tasa de Aprendizaje (Learning Rate 0.001): Es el tamaño del paso que el optimizador Adam utiliza para ajustar los pesos del modelo.

- 4. Resultado y Guardado:** El modelo final logra una precisión superior al 96% en los datos de prueba.

El modelo entrenado se guarda en un archivo llamado `modelo_clasificacion.pth` para su posterior uso en aplicaciones reales sin necesidad de reentrenar.

4.4. Código utilizado para el Modelo de Clasificación y enlaces

Se mostrarán algunas partes más relevantes del código ya que es extenso.

- **Código utilizado**

Conversion de Datos a Tensores PyTorch

```
X_train_tensor = torch.from_numpy(X_train).float().cuda()
y_train_tensor = torch.from_numpy(y_train).long().cuda()
X_test_tensor = torch.from_numpy(X_test).float().cuda()
y_test_tensor = torch.from_numpy(y_test).long().cuda()

print('Tipo de X_train_tensor:', X_train_tensor.dtype)
print('\n-----')
print('Dispositivo de X_train_tensor:', X_train_tensor)
```

```
Tipo de X_train_tensor: torch.float32
-----
Dispositivo de X_train_tensor: tensor([[0., 0., 0., ..., 0., 0., 0.],
      [0., 0., 0., ..., 0., 0., 0.],
      [0., 0., 0., ..., 0., 0., 0.],
      ...,
      [0., 0., 0., ..., 0., 0., 0.],
      [0., 0., 0., ..., 0., 0., 0.],
      [0., 0., 0., ..., 0., 0., 0.]], device='cuda:0')
```

Definicion de la Arquitectura de la Red Neuronal (Deep MLP)

```
# Definir los tamaños correctos para el dataset de billetes
input_size = 64*64*3 # 12288 características (píxeles)
```

```

output_size = 5      # 5 clases de billetes (0, 1, 2, 3, 4)

# Crear el modelo secuencial (Deep MLP con 8 capas ocultas)
model = nn.Sequential(
    # 8 Capas Ocultas
    nn.Linear(input_size, 8192), # Capa 1
    nn.ReLU(),
    nn.Linear(8192, 4096),        # Capa 2
    nn.ReLU(),
    nn.Linear(4096, 2048),        # Capa 3
    nn.ReLU(),
    nn.Linear(2048, 1024),        # Capa 4
    nn.ReLU(),
    nn.Linear(1024, 512),         # Capa 5
    nn.ReLU(),
    nn.Linear(512, 256),          # Capa 6
    nn.ReLU(),
    nn.Linear(256, 128),          # Capa 7
    nn.ReLU(),
    nn.Linear(128, 64),           # Capa 8
    nn.ReLU(),
    # Capa de Salida
    nn.Linear(64, output_size)   # Capa 9 (5 neuronas para
    las clases)
).to(torch.device("cuda")) # Mover el modelo a La GPU

print('Arquitectura del modelo Deep MLP:')
print(model)

```

Configuración del Entrenamiento (Loss y Optimizador)

```

# definir la funcion de perdida (CrossEntropy para
clasificacion multiclase)
criterion = nn.CrossEntropyLoss()

# definir el optimizador (adam con learning rate de 0.001)
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

print('Funcion de perdida:', criterion)
print('Optimizador:', optimizer)

```

```

Funcion de perdida: CrossEntropyLoss()
Optimizador: Adam (
Parameter Group 0
  amsgrad: False
  betas: (0.9, 0.999)
  capturable: False
  decoupled_weight_decay: False
  differentiable: False
  eps: 1e-08
  foreach: None
  fused: None
  lr: 0.001
  maximize: False
  weight_decay: 0
)

```

Bucle de Entrenamiento de la Red Neuronal

```

# Hiperparámetros
num_epochs = 2000 # epocas o siglos
log_interval = 100 # mostrar de cuanto en cuanto

# Listas para guardar métricas
train_losses = []
test_accuracies = []

# Bucle de entrenamiento
for epoch in range(1, num_epochs + 1):
    # Modo entrenamiento
    model.train()

    # Forward (adelante)
    outputs = model(X_train_tensor)
    loss = criterion(outputs, y_train_tensor) # calcular la
funcion de perdida

    # Backward (atras) y optimización
    optimizer.zero_grad() # Limpiar gradientes anteriores
    loss.backward()       # Calcular gradientes
    optimizer.step()       # Actualizar pesos

    # Guardar pérdida
    train_losses.append(loss.item())

    # Evaluar cada log_interval épocas # _ nos vota dos
valores solo utilizamos 1
    if epoch % log_interval == 0:

```

```

        test_accuracy, _ = evaluate(model, X_test_tensor,
y_test_tensor)
        test accuracies.append(test_accuracy)
        print(f'Epoch [{epoch}/{num_epochs}], Loss:
{loss.item():.4f}, Test Accuracy: {test_accuracy:.4f}')

# Guardar resultados
Adam_train_losses = train_losses.copy()
Adam_test accuracies = test accuracies.copy()

```

```

Epoch [100/2000], Loss: 1.0321, Test Accuracy: 0.4933
Epoch [200/2000], Loss: 0.5649, Test Accuracy: 0.7025
Epoch [300/2000], Loss: 0.5551, Test Accuracy: 0.6992
Epoch [400/2000], Loss: 0.6172, Test Accuracy: 0.6425
Epoch [500/2000], Loss: 0.2402, Test Accuracy: 0.8892
Epoch [600/2000], Loss: 0.7774, Test Accuracy: 0.6083
Epoch [700/2000], Loss: 0.0096, Test Accuracy: 0.9600
Epoch [800/2000], Loss: 0.0005, Test Accuracy: 0.9625
Epoch [900/2000], Loss: 0.0001, Test Accuracy: 0.9608
Epoch [1000/2000], Loss: 0.0000, Test Accuracy: 0.9608
Epoch [1100/2000], Loss: 0.0000, Test Accuracy: 0.9608
Epoch [1200/2000], Loss: 0.0000, Test Accuracy: 0.9608
Epoch [1300/2000], Loss: 0.0000, Test Accuracy: 0.9592
Epoch [1400/2000], Loss: 0.0000, Test Accuracy: 0.9583
Epoch [1500/2000], Loss: 0.0000, Test Accuracy: 0.9592
Epoch [1600/2000], Loss: 0.0000, Test Accuracy: 0.9592
Epoch [1700/2000], Loss: 0.0000, Test Accuracy: 0.9592
Epoch [1800/2000], Loss: 0.0000, Test Accuracy: 0.9592
Epoch [1900/2000], Loss: 0.0000, Test Accuracy: 0.9600
Epoch [2000/2000], Loss: 0.0000, Test Accuracy: 0.9600

```

```

# evaluacion final del modelo
final_accuracy, final_predictions = evaluate(model,
X_test_tensor, y_test_tensor)
print(f'\Accuracy final en el test: {final_accuracy:.4f}')

# mostrar algunas predicciones vs reales
print('\Algunas predicciones vs valores reales:')
for i in range(20):
    print(f'Prediccion" {final_predictions[i].item()}, Real:
{y_test_tensor[i].item()}')

```

```

\Accuracy final en el test: 0.9600
n\Algunas predicciones vs valores reales:
Prediccion" 1, Real: 1
Prediccion" 1, Real: 1
Prediccion" 0, Real: 0
Prediccion" 1, Real: 1
Prediccion" 3, Real: 3
Prediccion" 1, Real: 1
Prediccion" 4, Real: 4
Prediccion" 1, Real: 1
Prediccion" 0, Real: 0
Prediccion" 2, Real: 2

```


- **Código utilizado**

Drive

<https://drive.google.com/file/d/12aB1MXsbgAjRcdzQ5LXI2W1CvZgzcrLS/view?usp=sharing>

GitHub

https://github.com/EverQT/SIS420-IA/blob/main/Proyecto_Billetes/modelo_clasificacion.ipynb

5. Modelo YOLOv8n

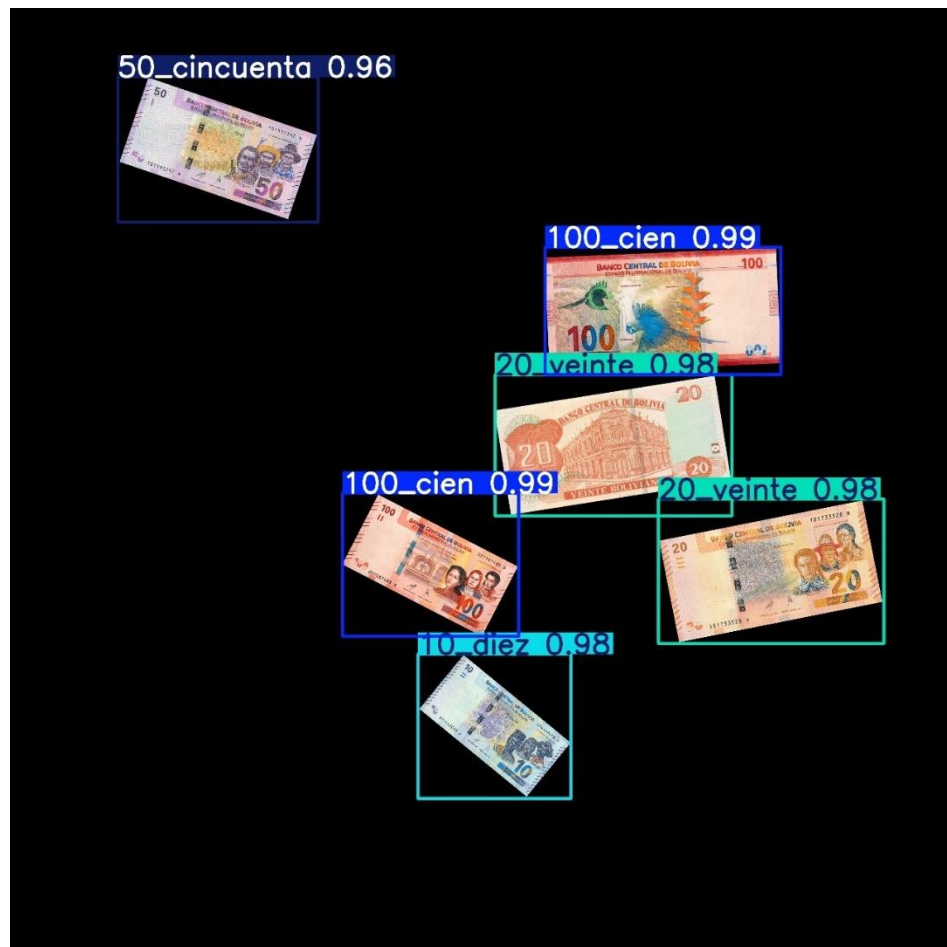
YOLOv8n es un modelo de visión computacional que detecta, clasifica y segmenta objetos en imágenes o videos en tiempo real. Se usa para que una computadora pueda reconocer automáticamente qué hay en una imagen, desde personas y autos hasta productos o billetes.

Lista de lo que necesitas para usar YOLOv8n

- Python 3.7 o superior
- **Paquete Ultralytics**
- GPU NVIDIA con CUDA (opcional, pero acelera mucho el entrenamiento y la inferencia)

5.1. Label Studio

Label Studio es la herramienta que usamos para encerrar objetos en imágenes y darles una etiqueta de clase. Con esas anotaciones se genera el dataset que luego YOLOv8n utiliza para aprender a detectar automáticamente esos objetos, en nuestro caso etiquetamos los billetes.

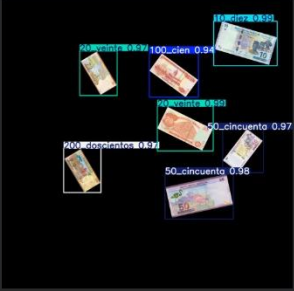


5.2. Notebook Colab personalizado

Se cargará el conjunto de datos etiquetados (billetes) para entrenar un modelo personalizado para la detección de los billetes se utiliza YOLOv8

```
image 15/21 /content/data/validation/images/93fff973-escena_066.png: 640x640 2 100_cien, 2 200_doscientos, 1 20_veinte, 7.2ms
image 16/21 /content/data/validation/images/bf897e25-escena_099.png: 640x640 1 100_cien, 1 10_diez, 1 20_veinte, 1 50_cincuenta, 7.2ms
image 17/21 /content/data/validation/images/c6384f9d-escena_087.png: 640x640 1 100_cien, 1 10_diez, 1 200_doscientos, 2 20_veintes, 2 50_cincuentas, 7.2ms
image 18/21 /content/data/validation/images/d99de793-escena_047.png: 640x640 2 10_diez, 1 20_veinte, 1 50_cincuenta, 7.2ms
image 19/21 /content/data/validation/images/dbfd2862-escena_009.png: 640x640 1 100_cien, 2 200_doscientos, 1 20_veinte, 2 50_cincuentas, 7.2ms
image 20/21 /content/data/validation/images/elf3bde4-escena_048.png: 640x640 1 100_cien, 2 10_diez, 1 200_doscientos, 1 20_veinte, 2 50_cincuentas, 8.1ms
image 21/21 /content/data/validation/images/f6f1661c-escena_068.png: 640x640 1 100_cien, 1 10_diez, 2 200_doscientos, 1 20_veinte, 2 50_cincuentas, 8.7ms
Speed: 3.8ms preprocess, 7.2ms inference, 2.5ms postprocess per image at shape (1, 3, 640, 640)
Results saved to /content/runs/detect/predict
Learn more at https://docs.ultralytics.com/modes/predict

import glob
from IPython.display import Image, display
for image_path in glob.glob(f'/content/runs/detect/predict/*.jpg')[:10]:
    display(Image(filename=image_path, height=400))
    print('\n')
```



5.3. Explicación de como se hizo el modelo YOLO

En esta parte para el entrenamiento de modelo de identificación de billetes, solo se uso un cuadernillo ya echo por otro desarrollador, es decir no se profundizo en como estaba echo, si no que se siguió los pasos o guía dada por el mismo desarrollador.

- **enlaces de video utilizado como guía para YOLOv8**

<https://youtu.be/r0RspiLG260?si=8JGq-94bAlxz3U3I>

- **enlaces Drive cuadernillo usado**

<https://drive.google.com/file/d/1EOYVpCU2Gvy6mNMk4Q20oXLNoLaj4amk/view?usp=sharing>

- **enlaces GitHub cuadernillo usado**

https://github.com/everqt/sis420-ia/blob/main/proyecto_billetes/train_yolo_models.ipynb

6. DroidCam



Es una aplicación que convierte tu teléfono Android o iPhone en una cámara web para tu computadora, se puede utilizar por Wifi y por USB en nuestro caso se usó USB

6.1. Código utilizado

Se realizó un Script para que un teléfono se usara como cámara, que sacara una foto después de 5 segundos

```
import cv2
import os
import time
from IPython.display import display, Image # Importaciones
útiles en un notebook

# 1. Configuración de la carpeta de destino
nombre_carpeta = "Fotos_Capturadas"
ruta_completa = os.path.join(os.getcwd(), nombre_carpeta)

if not os.path.exists(ruta_completa):
    os.makedirs(ruta_completa)
    print(f"Carpeta creada: {ruta_completa}")

# 2. Iniciar la cámara (¡MODIFICA ESTA LÍNEA!)
# --- CAMBIA ESTE NÚMERO Y VUELVE A EJECUTAR ---
INDICE_CELULAR = 2
# -----
cap = cv2.VideoCapture(INDICE_CELULAR)

# Verificar si la cámara se abrió correctamente
```

```

if not cap.isOpened():
    print(f"--- ERROR CRÍTICO: No se pudo abrir la cámara con el índice {INDICE_CELULAR}. ---")
    print("Prueba cambiando INDICE_CELULAR a otro valor (0, 1, 2, 3, etc.)")
    exit()

# Opcional: Establecer resolución
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)

print(f"--- Cámara iniciada con el índice: {INDICE_CELULAR} ---")
print("Mostrando vista previa durante 5 segundos...")

# Guardar el tiempo inicial
tiempo_inicio = time.time()
TIEMPO_ESPERA = 5 # Segundos

# Variables para gestionar la ventana de OpenCV
ventana_abierta = False

while True:
    ret, frame = cap.read()

    if not ret:
        print("Error al leer el frame. Conexión perdida.")
        break

    # Mostrar la imagen en una ventana
    cv2.imshow('Vista Previa - Celular USB (Se cierra en 5 segundos)', frame)
    ventana_abierta = True

    # Verificar si han pasado 5 segundos
    if time.time() - tiempo_inicio >= TIEMPO_ESPERA:
        nombre_foto = f"foto_{int(time.time())}.jpg"
        ruta_archivo = os.path.join(ruta_completa, nombre_foto)

        cv2.imwrite(ruta_archivo, frame)
        print(f"¡Foto guardada en: {ruta_archivo}!")
        display(Image(ruta_archivo))

        break # Salir del bucle

```

```
cv2.waitKey(1)

# Liberar la cámara y cerrar ventanas
cap.release()
if ventana_abierta:
    cv2.destroyAllWindows()

print("Programa finalizado.")
```

- Código utilizado

Drive

<https://drive.google.com/file/d/1H8usmAdU7yAfH9Hezp53ZGeGSE9QPxX8/view?usp=sharing>

GitHub

https://github.com/everqt/sis420-ia/blob/main/proyecto_billetes/principal/identificador.ipynb

7. Identificador

se utiliza el modelo yolo ya entrenado previamente para que, una vez tomada la foto, este pueda capturar y recortar los billetes que hay en la foto.



1. Imágenes detectadas

Utilizando el modelo entrenado se detectan las imágenes de los billetes y se los recorta



2. Redimensionan a 64x64

Se los redimensiona porque el dataset de **Modelo_Clasificacion** utiliza ese tipo de datos



3. Conversión a dataset CSV

Se convierte las imágenes a csv para poder utilizar el modelo entrenado Modelo_Clasificacion para así poder hacer las predicciones

```
import cv2
import os
import numpy as np
import csv
import torch

# ... (Configuración de carpetas omitida para brevedad) ...
input_folder = "imagenes_64x64"
output_dir = "dataset_detecciones"
output_csv = os.path.join(output_dir, "imagenes_dataset.csv")
os.makedirs(output_dir, exist_ok=True)

data = []
file_names = []

# Iterar sobre todas las imágenes en la carpeta
for filename in os.listdir(input_folder):
    if filename.lower().endswith((".jpg", ".jpeg", ".png",
    ".bmp")):
        img_path = os.path.join(input_folder, filename)
        img = cv2.imread(img_path)

        if img is None:
            continue

        # Redimensionar a 64x64 (Manteniendo BGR)
        img_resized = cv2.resize(img, (64, 64))

        # 🌀 PASO DE CORRECCIÓN CRÍTICO: CONVERTIR DE BGR
        (OpenCV) A RGB
        # ¡Esto asegura la consistencia con tu modelo entrenado!
```

```

        img_rgb = cv2.cvtColor(img_resized, cv2.COLOR_BGR2RGB) #
¡ESTA LÍNEA ES CLAVE!

        # NORMALIZACIÓN (¡Crucial para el modelo MLP!)
        # Ahora normalizamos la matriz RGB
        img_normalized = img_rgb / 255.0

        # Convertir a vector (Ahora es un vector de orden RGB)
        img_flat = img_normalized.flatten()

        # Agregar al dataset
        data.append(img_flat)
        file_names.append(filename)

# Convertir lista a array NumPy y guardar en CSV
data_array = np.array(data)
np.savetxt(output_csv, data_array, delimiter=",", fmt="%.6f")

print(f"✅ Dataset generado en {output_csv} con forma
{data_array.shape} y orden de canales RGB.")

```

4. Proceso para la realiza las predicciones

Se realiza un proceso con el CSV de las nuevas imágenes para poder hacer las predicciones, se usan:

- Importación de librerías necesarias
- La carga de dataset CSV
- Se normaliza

```

# normalizar los valores de pixeles entre 0 y 1
#X_normalized = X / 255.0 # valor del RGB rojo, azul y
amarillo

X_normalized = X

print('Primeros 5 valores de píxel normalizados:',
X_normalized[0, :5])

print('Valores minimos y maximos despues de normalizar:')
print('Minimo:', X_normalized.min())
print('Maximo:', X_normalized.max())

```

- Se convierte a tensores

```
#X_tensor = torch.tensor(X_normalized,
dtype=torch.float32).unsqueeze(0)
#print(X_tensor)

# Solución al error de dimensión
X_tensor = torch.tensor(X_normalized, dtype=torch.float32)
# <-- ¡Eliminar .unsqueeze(0)!
print('Forma esperada para el modelo:', X_tensor.shape)
```

- Se define la red neuronal

```
import torch.nn as nn # Asegurarse de tener la librería
importada

# Definir los tamaños correctos para el dataset de
billetes
input_size = 64*64*3 # 12288 características (píxeles)
output_size = 5      # 5 clases de billetes (0, 1, 2, 3, 4)

# Crear el modelo secuencial (Deep MLP con 8 capas
ocultas)
model = nn.Sequential(
    # 8 Capas Ocultas
    nn.Linear(input_size, 8192), # Capa 1
    nn.ReLU(),
    nn.Linear(8192, 4096),        # Capa 2
    nn.ReLU(),
    nn.Linear(4096, 2048),        # Capa 3
    nn.ReLU(),
    nn.Linear(2048, 1024),        # Capa 4
    nn.ReLU(),
    nn.Linear(1024, 512),         # Capa 5
    nn.ReLU(),
    nn.Linear(512, 256),          # Capa 6
    nn.ReLU(),
    nn.Linear(256, 128),          # Capa 7
    nn.ReLU(),
    nn.Linear(128, 64),           # Capa 8
    nn.ReLU(),
    # Capa de Salida
    nn.Linear(64, output_size)   # Capa 9 (5 neuronas para
las clases)
).to(torch.device("cpu")) # Mover el modelo a la GPU
```

```

model.load_state_dict(torch.load('modelo_clasificacion.pth',
map_location='cpu')) # ← Añade esta línea después

print('Arquitectura del modelo Deep MLP:')
print(model)

```

- Se realiza la inferencia de los billetes recortados

```

# Inferencia final sobre tus billetes recortados
(CORREGIDO)
model.eval()
with torch.no_grad():
    outputs = model(X_tensor)
    _, predicted_tensor = torch.max(outputs, 1) # Usamos
    un nombre temporal

# 🌀 CORRECCIÓN: Convertir el tensor de predicciones a una
    lista simple de Python (escalares)
predicted_indices = predicted_tensor.tolist()

# Mapeo de clases
#valores = [200, 100, 50, 20, 10]
#valores = [10, 20, 50, 100, 200]
valores = [0, 1, 2, 3, 4]
#valores = [4, 3, 2, 1, 0]

# Mostrar resultados
print("\n=== PREDICCIONES FINALES ===")
archivos = sorted([f for f in os.listdir("imagenes_64x64")
if f.endswith(".jpg")])

# Iteramos sobre la lista de índices corregida
for i, pred_index in enumerate(predicted_indices):
    # pred_index ahora es un entero (int), sin error
    .item()
    monto = valores[pred_index]

    # Mantenemos el formato de salida
    print(f"{archivos[i]:15} → {monto} Bolivianos (Índice:
{pred_index})")

print("=====")

```

- **Resultados**

```
=== PREDICCIONES FINALES ===
imagen000.jpg → 3 Bolivianos (Índice: 3)
imagen001.jpg → 3 Bolivianos (Índice: 3)
=====
```

```
=== RESULTADO FINAL ===
imagen000.jpg → CLASE 3
imagen001.jpg → CLASE 3
=====
```

```
=== TENSOR DE RESULTADO COMPLETO (LOGITS/SCORES) ===
tensor([[ -45.4150,  87.0520, -19.9225,  89.8458,  47.7331],
        [-52.3346,  95.2603, -11.9421, 119.2609,  53.9893]])
Forma del Tensor: torch.Size([2, 5])
=====

=== ÍNDICES DE CLASE PREDICHOS ===
imagen000.jpg → CLASE 3
imagen001.jpg → CLASE 3
=====
```

- **Interpretación**

La foto tenía dos billetes de 100, el modelo yolo reconoció los dos billetes los recorto cada uno por separado, se trató las imágenes desde el redimensionamiento a 64x64, pasado la conversión a CSV, normalización, etc. Hasta el punto que se hizo la inferencia en el modelo entrenado de Modelo_Clasicacion, este hizo una predicción de clase 3:

Clase 0 → 10 Bs

Clase 1 → 20 Bs

Clase 2 → 50 Bs

Clase 3 → 100 Bs

Clase 4 → 200 Bs

Se puede decir que modelo hizo una predicción acertada con los billetes detectados que eran de 100 bs.

- **Código utilizado**

Drive

<https://drive.google.com/file/d/1H8usmAdU7yAfH9Hezp53ZGeGSE9QPxX8/view?usp=sharing>

GitHub

https://github.com/EverQT/SIS420-IA/blob/main/Proyecto_Billetes/principal/identificador.ipynb

8. Enlaces del proyecto

- **Drive**

<https://drive.google.com/drive/folders/1HyTWbynJnMSs9PHzPnYq1UkGEGDpgEw6?usp=sharing>

- **GitHub**

https://github.com/EverQT/SIS420-IA/tree/main/Proyecto_Billetes